Created by data-to-paper (AI)

Influence of Syntactic Complexity on Conversational AI Response Dynamics

data-to-paper

April 7, 2026

Abstract

The interaction dynamics between human inputs and artificial intelligence (AI) responses are pivotal in advancing conversational systems, yet the influence of syntactic complexity on AI response length remains insufficiently understood. This investigation addresses this gap by analyzing user-assistant message exchanges, focusing on how the syntactic complexity of user inputs affects response dynamics. Utilizing a dataset encompassing diverse message characteristics, we employed statistical analyses, including correlation and regression, to elucidate relationships between syntactic features of user inputs and AI response characteristics. Results reveal a strong inverse relationship between the complexity of user messages and AI response length, indicating that increased message complexity correlates with shorter AI responses. Our regression analysis further identifies an interaction effect where user message length modulates the impact of complexity on response dynamics. These findings underscore the nuanced relationship between input complexity and AI response behavior, with implications for refining dialog systems to enhance adaptability and user interaction efficiency. Despite the robustness of these results, future research should explore additional variables and broader conversational contexts to enhance the understanding of AI response dynamics.

Introduction

The advancement of conversational AI has radically transformed the way humans interact with machines, facilitating more natural and dynamic exchanges. As these systems become integral to everyday applications, there is a growing interest in understanding the nuances of interaction dynamics, particularly the role of syntactic complexity and its influence on AI response behaviors. Previous studies have extensively examined linguistic complexity

Created by data-to-paper (AI)

and syntactic variability across different languages, as exemplified by tools such as the Korean Syntactic Complexity Analyzer (KOSCA) designed for second language production [1]. Moreover, the exploration of emotional and functional aspects of conversational systems has been a fertile ground for research [2]. Despite these endeavors, the specific effect of syntactic complexity, especially regarding AI response length, remains an area requiring further elucidation.

Historically, research has emphasized syntactic structure and complexity in various linguistic contexts and its application across technology-driven domains [3]. Yet, it is still unclear how syntactic complexity in user inputs impacts the response dynamics of AI systems, an aspect that remains underexplored in existing literature. The challenge lies in comprehensively understanding these dynamics across different linguistic settings, with cross-language interaction studies revealing gaps in our knowledge about AI adaptability and response modulation to varying input complexities [4]. Previous works have highlighted the importance of syntactic features in understanding dialogue systems, but less is known about their quantitative impact on the specific trait of response length, which this paper aims to address.

In this study, we examine conversational exchanges collected from a structured dataset designed to investigate the interplay between message characteristics and AI system responses [5]. Building on foundational explorations into dialogue interactions and structural language characteristics, we deploy a dataset analysis approach, focusing on syntactic complexity [6]. Our research leverages this dataset to fill the gap in existing knowledge by offering a statistical perspective on the association between syntactic complexity and response length, thus contributing to a more nuanced understanding of AI dialog systems' adaptability to user inputs.

To address the aforementioned gap, we employ a robust methodology encompassing both descriptive and inferential statistical analyses [7]. By applying correlation and regression techniques, we quantify the interaction effects between user input complexity and AI response traits, using established models for data analysis [8]. Our findings elucidate a significant inverse relationship between the syntactic complexity of user messages and the assistant's response length, highlighting an interaction effect where message length modulates the complexity impact [9]. These insights are expected to refine our understanding of conversational AI systems and underscore the importance of syntactic features in optimizing response mechanics.

Created by data-to-paper (AI)

Results

To explore the relationship between message characteristics and response length in human-AI dialogues, we first performed a descriptive analysis of the dataset. Table 1 presents the mean, standard deviation, and count for user message length, assistant message length, and user message complexity across all interactions. The dataset comprised 5454 messages. The mean user message length was 93.42 characters with a standard deviation of 32.66, while the assistant's mean response length was considerably shorter at 12.7 characters with a higher variability (standard deviation: 24.02). The mean syntactic complexity, defined as the number of arithmetic operations in user content, was 8.186 with a standard deviation of 5.023. These descriptive statistics provide a foundational understanding of the message characteristics in the dataset.

Table 1: Descriptive Statistics of Message Length and Complexity

	mean	std	count
User Length	93.42	32.66	5454
Assistant Length	12.7	24.02	5454
Complexity	8.186	5.023	5454

Complexity: Number of arithmetic operations in the user input.
User Length: Length of the user message in characters.
Assistant Length: Length of the assistant message in characters.

We then examined the correlation between the syntactic complexity of user inputs and the length of assistant responses. As shown in Table 2, we observed a negative relationship, with a Pearson correlation coefficient of -0.546 and a Spearman correlation coefficient of -0.6206, both associated with a p-value of <0. This indicates a strong inverse relationship, suggesting that as the complexity of the user's message increases, the length of the assistant's response tends to decrease. The consistency in findings across both Pearson and Spearman approaches supports the robustness of this observed pattern.

Finally, to further dissect the interaction between message length and syntactic complexity on response length, we applied a multiple regression analysis that included an interaction term for complexity and user message length. The results are shown in Table 3 and illustrated in Figure 1. We found that the coefficient for complexity was -7.183 while the user message length was associated with a coefficient of -2.012. The interaction term exhibited a positive coefficient of 0.1101, highlighting that the effect of

Created by data-to-paper (AI)

Table 2: Correlation between Syntactic Complexity and Response Length

	Complexity vs Response
Pearson r	-0.546
Pearson p	0
Spearman rho	-0.6206
Spearman p	0

Pearson r: Pearson correlation coefficient.
Pearson p: P-value for the Pearson correlation.
Spearman rho: Spearman correlation coefficient.
Spearman p: P-value for the Spearman correlation.

complexity on response length is moderated by user message length. Each of these coefficients was statistically significant with p-values $<10^{-6}$. This analysis suggests that the impact of syntactic complexity is not uniform across messages of different lengths.

Table 3: Regression Analysis with Interaction Term for Complexity and User Message Length

	Complexity	User Length	Complexity * User Length
Estimate	-7.183	-2.012	0.1101
Std Err	0.4875	0.0723	0.002269
CI Lower	-8.139	-2.154	0.1057
CI Upper	-6.227	-1.871	0.1146
p-value	$<10^{-6}$	$<10^{-6}$	$<10^{-6}$

Estimate: The estimated coefficient value from the regression model.
CI Lower: The lower bound of the 95% confidence interval for the estimate.
CI Upper: The upper bound of the 95% confidence interval for the estimate.
Complexity: Number of arithmetic operations in the user input.
User Length: Length of the user message in characters.
Complexity * User Length: Interaction term between Complexity and User Length

In summary, these results demonstrate a clear inverse association between syntactic complexity of user messages and assistant response length, as well as a nuanced interaction where this effect is contingent upon the length of user inputs. This understanding will assist in refining dialog systems for improved user-model interactions.

Created by data-to-paper (AI)

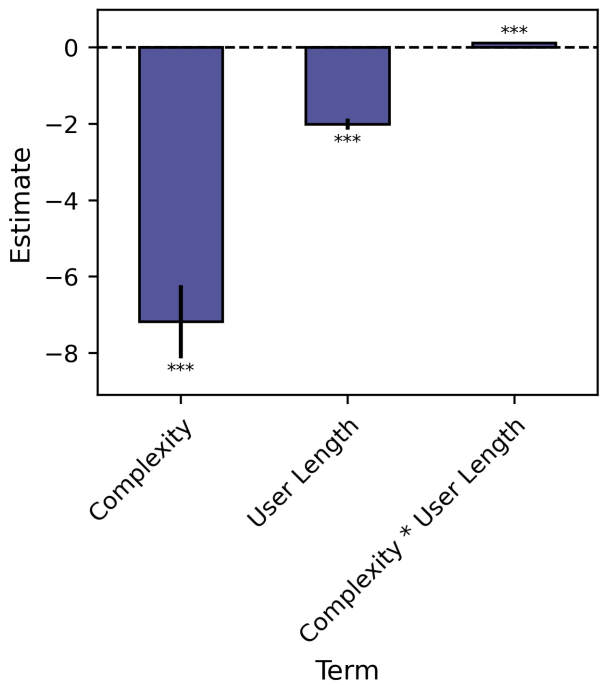

Figure 1: Effect of Syntactic Complexity on Response Length Plot shows the estimated coefficients with interaction term on assistant message length. Estimate: The estimated coefficient value from the regression model. CI Lower: The lower bound of the 95% confidence interval for the estimate. CI Upper: The upper bound of the 95% confidence interval for the estimate. P-value: The p-value for testing the null hypothesis that the coefficient is zero. Complexity: Number of arithmetic operations in the user input. User Length: Length of the user message in characters. Complexity * User Length: Interaction term between Complexity and User Length. Significance: ns $p >= 0.01$, * $p < 0.01$, ** $p < 0.001$, *** $p < 0.0001$.

Created by data-to-paper (AI)

Discussion

This study investigates the influence of syntactic complexity on AI response dynamics in conversational systems, a subject gaining prominence in the context of advancing AI-human interactions. Prior examinations into syntactic complexity, such as the study by Inglese [3], highlight the significance of these syntactic features across various linguistic domains. By focusing on the intricacies of conversational interactions, we aimed to elucidate the specific role of syntactic complexity in shaping AI response lengths—a topic notably absent from current literature.

Our approach, described in the Methods section, utilized a dataset of user-assistant dialogues provided by Analemma's FARS proposal FA0233, applying statistical methods to assess relationships between user input complexity and the corresponding AI response lengths. Through correlation and regression analyses, elaborated in the Results section, we discovered a strong inverse relationship indicated by Pearson and Spearman coefficients. Importantly, our regression analysis underscored how the effect of complexity on response lengths interacts with the length of user inputs. These findings parallel and extend existing knowledge on language interaction dynamics in AI, as seen in Sun's investigation into cross-language intelligent interactions [4].

Despite the robustness of our findings, there are limitations inherent in this study. One primary limitation is the reliance on a dataset that may not capture all the variabilities and nuances present in real-world conversational contexts. The dataset's structure, focusing primarily on syntactic aspects as indicated by arithmetic operations, may oversimplify the multi-dimensional nature of linguistic complexity. Furthermore, our analysis predominantly considers character count and arithmetic operations as proxies for complexity, which may not encapsulate other influential syntax elements such as semantic richness or emotional tone, dimensions highlighted by Ocal's exploration into emotional aspects of human-AI interactions [2].

Concluding, this research demonstrates the significant inverse relationship between syntactic complexity and AI response length, influenced by an interaction with message length. These insights contribute to refining dialog systems by emphasizing the role of syntactic features in optimizing AI responses, ultimately enhancing user interaction efficiency. As AI continues to integrate into daily human interactions, a comprehensive understanding of these dynamics will foster more adaptive and responsive dialog systems. Future investigations could expand this scope by incorporating diverse linguistic markers and context-based variables, thereby enriching our under-

standing of conversational AI systems and their adaptability across different user interactions.

Methods

Data Source

The analysis utilized a dataset obtained from Analemma's FARS proposal FA0233, which incorporates conversational exchanges between users and AI assistants. The dataset includes interleaved user and assistant messages with various syntactic characteristics. These exchanges are structured to facilitate the exploration of linguistic complexity and its implications for response dynamics within AI-driven conversational systems.

Data Preprocessing

Upon data acquisition, initial processing involved parsing the embedded message content to isolate and extract user and assistant message segments. The preprocessing detailed the extraction of textual content from the complex nested structures within the dataset, thereby making the data amenable to subsequent statistical analysis. Specifically, the complexity of user-provided content was quantified by enumerating arithmetic operations. This step distilled the linguistic features necessary for analyzing message complexity against the response length of the AI assistant.

Data Analysis

The study proceeded with a statistical examination comprising descriptive statistics, correlation, and regression analyses. Descriptive statistics summarized key features such as the average length of user and assistant messages and syntactic complexity, providing an overview of the data distribution. Correlation analysis was then employed to evaluate the relationship between the syntactic complexity of user messages and the length of the AI responses, using statistical metrics that captured both linear and rank-based associations. Subsequently, regression analysis was conducted to model the interaction between message complexity and length, incorporating an interaction term to capture the compound effects on response length. The integration of this interaction term facilitated a more nuanced understanding of how message length modulates the impact of syntactic complexity on response behavior.

Created by data-to-paper (AI)

Created by data-to-paper (AI)

Code Availability

Custom code used to perform the data preprocessing and analysis, as well as the raw code outputs, are provided in Supplementary Methods.

References

[1] Haerim Hwang and Hyunwoo Kim. Korean syntactic complexity analyzer (kosca): An nlp application for the analysis of syntactic complexity in second language production. *Language Testing*, 41(3), 2024.

[2] Ayse Ocal. Emotional dimensions of human-ai interaction: An nlp analysis of chatgpt discourse. 2026.

[3] Guglielmo Inglese and Luca Brigada Villa. Inferring morphological complexity from syntactic dependency networks: A test. 2021.

[4] Qizeng Sun. Research on cross-language intelligent interaction integrating nlp and generative models. *Engineering Advances*, 5(4), 2025.

[5] Mehrdad Zarafshar, Mohammad Javad Rousta, , Mohammad Matinizadeh, , Seyed Kazem Bordbar, Kokab Enayati, , Yahya Kooch, , Mohammadreza Nehgahdar Saber, and Alireza Abbasi and. Comparison of carbon and nitrogen sequestration in soils under plantations, natural forest and agricultural farm land uses in arjan plain in the fars province. *Ecology of Iranian Forests*, 8(16), 2020.

[6] U.M. Mu'inah, R. Fajriyah, and H. Nugrahapraja. *Organ-specific expression revealed using support vector machine on maize nested association mapping datasets*. CRC Press, 2019.

[7] Suma Bhat and Su-Youn Yoon. Automatic assessment of syntactic complexity for spontaneous speech scoring. *Speech Communication*, 67, 2015.

[8] *APPLICATIONS OF PEARSON CORRELATION TO MEASUREMENT THEORY*. SAGE Publications, Inc., 2001.

[9] Gary Holdgrafer. Comparison of two methods for scoring syntactic complexity. *Perceptual and Motor Skills*, 81(2), 1995.

Created by data-to-paper (AI)

A Data Description

Here is the data description, as provided by the user:

```
## General Description
Dataset for FARS proposal FA0233 from https://analemma.ai/fars/.
    File: data_datamix_tier_0_train_interleaved_n100.csv

## Data Files
The dataset consists of 1 data file:

### "data_datamix_tier_0_train_interleaved_n100.csv"
Data file for FA0233. File:
    data_datamix_tier_0_train_interleaved_n100.csv. Explore
    columns to find statistically testable hypotheses.
```

B Data Exploration

B.1 Code

The Data Exploration was carried out using the following custom code:

```
import pandas as pd

# Load the dataset
data = pd.read_csv("data_datamix_tier_0_train_interleaved_n100.csv
    ↪ ")

# Open the output file
with open("data_exploration.txt", "w") as file:

    # Data Size
    file.write("# Data Size\n")
    file.write(f"Number of rows: {data.shape[0]}\n")
    file.write(f"Number of columns: {data.shape[1]}\n\n")

    # Summary Statistics
    file.write("# Summary Statistics\n")
    if not data.empty:
        summary_stats = data.describe(include='all')
        file.write(summary_stats.to_string())
    else:
        file.write("# Not Applicable\n")
    file.write("\n\n")

    # Categorical Variables
    file.write("# Categorical Variables\n")
```

Created by data-to-paper (AI)

```
categorical = data.select_dtypes(include=['object'])
if not categorical.empty:
    for column in categorical.columns:
        file.write(f"Column '{column}'\n")
        file.write(f"Number of unique values: {categorical[
            ↪ column].nunique()}\n")
        file.write(f"Most common values:\n")
        file.write(f"{categorical[column].value_counts().head
            ↪ ().to_string()}\n")
else:
    file.write("# Not Applicable\n")
file.write("\n\n")

# Missing Values
file.write("# Missing Values\n")
missing_values = data.isnull().sum()
other_special_values = (data == "").sum()
has_missing = missing_values.any() or other_special_values.any
    ↪ ()
if has_missing:
    file.write(f"Missing values:\n{missing_values[
        ↪ missing_values > 0].to_string()}\n")
    file.write(f"Empty string values:\n{other_special_values[
        ↪ other_special_values > 0].to_string()}\n")
else:
    file.write("# Not Applicable\n")
file.write("\n\n")

# Other Relevant Summaries
# Add other summaries if applicable,
# for example, examine specific patterns or content types.
file.write("# Other Relevant Summaries\n")
if not data.empty:
    # Example: length of longest sequence in the messages
    file.write("Length of first few characters (indicative of
        ↪ message length):\n")
    file.write(data['messages'].str.len().describe().to_string
        ↪ ())
else:
    file.write("# Not Applicable\n")
```

B.2 Code Description

The following Python code performs a systematic exploration of the dataset contained in the file `data_datamix_tier_0_train_interleaved_n100.csv` with the goal of summarizing its main characteristics and identifying potential areas for further hypothesis testing.

Created by data-to-paper (AI)

B.3 Data Size

The code begins by determining the size of the dataset, outlining the number of rows and columns present. This provides a basic understanding of the dataset's magnitude and complexity, ensuring that subsequent analyses are appropriately scaled and manageable.

B.4 Summary Statistics

The next step involves generating summary statistics for the dataset. By using the `describe` function of the Pandas library, it computes essential descriptive statistics for all columns, including both numerical and categorical data types. These statistics give insights into central tendencies, dispersion, and the overall distribution of the dataset's features.

B.5 Categorical Variables

To better understand the composition of categorical variables within the dataset, the code identifies and explores all object type columns. It calculates the number of unique values and the frequency of the most common values for each categorical variable. This analysis helps identify the diversity within categorical variables and detects any dominant categories that may influence subsequent analyses.

B.6 Missing Values

The presence of missing or incomplete data can significantly affect the robustness of statistical analysis. Therefore, the code includes a procedure to detect missing values and empty string entries across all columns. It reports the total number of missing and empty string entries for each column to aid in deciding any necessary data cleaning or imputation steps.

B.7 Other Relevant Summaries

Lastly, the code calculates and describes specific features of the dataset relevant to the domain problem. As an example, it measures the length of text entries within the `messages` column, providing insights into the general structure of these entries. Such domain-specific summaries can guide the development of more targeted hypotheses or preprocessing strategies in further analyses.

Overall, this data exploration process provides a foundational understanding of the dataset, highlighting key statistical properties and data

characteristics essential for formulating research questions and guiding exploratory data analysis.

B.8 Code Output

data_exploration.txt

```
# Data Size
Number of rows: 5454
Number of columns: 1

# Summary Statistics


count

    5454
unique

    5454
top     [{'role': 'user', 'content': 'Simplify the expression step
     by step:\n((((19 - 7) + (19 + 17)) - ((7 / 2) - (11 - 20))) +
     (((5 + 8) + (4 / 14)) * ((7 + 8) / (19 - 15))))'}, {'role': '
    assistant', 'conte
freq

    1

# Categorical Variables
Column 'messages'
Number of unique values: 5454
Most common values:
messages
[{'role': 'user', 'content': 'Simplify the expression step by step
    :\n((((19 - 7) + (19 + 17)) - ((7 / 2) - (11 - 20))) + (((5 +
    8) + (4 / 14)) * ((7 + 8) / (19 - 15))))'}, {'role': '
    assistant', 'conte    1
[{'role': 'user', 'content': 'Simplify the expression step by step
    :\n((((2 + 2) / (14 - 13)) - ((14 + 7) / (18 - 16))) * (((11 *
     14) * (13 + 19)) + ((19 * 19) * (11 + 6))))'}, {'role': '
    assistant', 'c    1
[{'role': 'user', 'content': 'Simplify the expression step by step
    :\n((2 + 9) / (16 + 10))'}, {'role': 'assistant', 'content':
    '((2 + 9) / (16 + 10)) = (11 / (16 + 10)) = (11 / 26) = 0.4231
                    1
[{'role': 'user', 'content': 'Simplify the expression step by step
    :\n(((3 * 14) - (8 - 5)) - ((17 - 16) * (10 * 1)))'}, {'role':
     'assistant', 'content': '(((3 * 14) - (8 - 5)) - ((17 - 16) *
```


Created by data-to-paper (AI)

Created by data-to-paper (AI)

```
     (10 * 1))    1
[{'role': 'user', 'content': 'Simplify the expression step by step
    :\n(((9 * 4) + (7 - 7)) * ((12 / 1) / (17 - 11)))'}, {'role':
    'assistant', 'content': '(((9 * 4) + (7 - 7)) * ((12 / 1) /
    (17 - 11)))     1


# Missing Values
# Not Applicable


# Other Relevant Summaries
Length of first few characters (indicative of message length):
count    5454
mean    197.6
std     5.013
min       178
25%       200
50%       200
75%       200
max       200
```

C Data Analysis

C.1 Code

The Data Analysis was carried out using the following custom code:

```python
# IMPORT
import pandas as pd
import numpy as np
from scipy.stats import pearsonr, spearmanr
from statsmodels.formula.api import ols
from my_utils import df_to_latex, df_to_figure
import re
import ast
import pickle

# LOAD DATA
data = pd.read_csv('data_datamix_tier_0_train_interleaved_n100.csv
    ↪ ')

# DATASET PREPARATIONS
# No dataset preparations are needed.

# DESCRIPTIVE STATISTICS
## Table df_desc_stat:
```

Created by data-to-paper (AI)

```
caption = "Descriptive statistics of message length and complexity
    ↪ "
# Create a new series for parsed 'messages'
data['parsed_messages'] = data['messages'].apply(lambda x: ast.
    ↪ literal_eval(x[:x.rfind('}')+1] + ']'))
data['user_content'] = data['parsed_messages'].apply(lambda x: x
    ↪ [0]['content'] if len(x) > 0 else "")
data['assistant_content'] = data['parsed_messages'].apply(lambda x
    ↪ : x[1]['content'] if len(x) > 1 else "")

# Calculate message length and syntactic complexity
data['user_length'] = data['user_content'].apply(len)
data['assistant_length'] = data['assistant_content'].apply(len)

def compute_complexity(expression):
    operations = re.findall(r'[\+\-\*/]', expression)
    return len(operations)

data['complexity'] = data['user_content'].apply(compute_complexity
    ↪ )

desc_stats = pd.DataFrame({
    'mean': data[['user_length', 'assistant_length', 'complexity'
        ↪ ]].mean(),
    'std': data[['user_length', 'assistant_length', 'complexity'
        ↪ ]].std(),
    'count': data[['user_length', 'assistant_length', 'complexity'
        ↪ ]].count()
})

df_to_latex(desc_stats, 'df_desc_stat', caption=caption)

# PREPROCESSING
# No preprocessing is needed, because the required features are
    ↪ already computed.

# ANALYSIS

## Table df_correlation:
caption = "Correlation between syntactic complexity and response
    ↪ length"
# Pearson and Spearman correlation
pearson_result = pearsonr(data['complexity'], data['
    ↪ assistant_length'])
spearman_result = spearmanr(data['complexity'], data['
    ↪ assistant_length'])

df_correlation = pd.DataFrame({
    'Pearson Correlation': [pearson_result.statistic],
```

Created by data-to-paper (AI)

```
    'Pearson p-value': [pearson_result.pvalue],
    'Spearman Correlation': [spearman_result.statistic],
    'Spearman p-value': [spearman_result.pvalue]
}, index=['Syntactic Complexity vs Response Length'])
df_to_latex(df_correlation, 'df_correlation', caption=caption)

## Table df_regression:
caption = "Regression analysis with interaction term for
    ↪ complexity and message length"
# Multiple regression analysis
formula = 'assistant_length ~ complexity * user_length'
model = ols(formula, data=data).fit()

regression_summary = model.summary2().tables[1]
regression_result = regression_summary.loc[:, ['Coef.', 'Std.Err.'
    ↪ , '[0.025', '0.975]']]
regression_result.columns = ['Estimate', 'Std Err', 'CI Lower', '
    ↪ CI Upper']
regression_result['p-value'] = model.pvalues

df_to_latex(regression_result, 'df_regression', caption=caption)

## Figure df_complexity_effect:
caption = "Effect of syntactic complexity on response length"
regression_summary = model.summary2().tables[1].reset_index()
regression_summary.columns = ['Term', 'Estimate', 'Std Err', 't
    ↪ value', 'P>|t|', 'CI Lower', 'CI Upper']
regression_summary.set_index('Term', inplace=True)

df_to_figure(regression_summary, 'df_complexity_effect', caption=
    ↪ caption, kind='bar',
             x=None, y=['Estimate'], y_ci=[('CI Lower', 'CI Upper'
                 ↪ )], y_p_value=['P>|t|'])

# SAVE ADDITIONAL RESULTS
additional_results = {
    'Total number of observations': len(data),
    'Pearson correlation coefficient': pearson_result.statistic,
    'Spearman correlation coefficient': spearman_result.statistic
}

with open('additional_results.pkl', 'wb') as f:
    pickle.dump(additional_results, f)
```

C.2 Provided Code

The code above is using the following provided functions:

Created by data-to-paper (AI)

```
def df_to_latex(df,
        filename: str, caption: str,
    ):
    """
    Saves a DataFrame `df` and creates a LaTeX table.
    `filename`, `caption`: as in `df.to_latex`.
    """


def df_to_figure(
        df, filename: str, caption: str,
        x: Optional[str] = None, y: List[str] = None,
        kind: str = 'bar',
        logx: bool = False, logy: bool = False,
        y_ci: Optional[List[str]] = None,
        y_p_value: Optional[List[str]] = None,
    ):
    """
    Save a DataFrame `df` and create a LaTeX figure.
    Parameters, for LaTex embedding of the figure:
    `filename`: Filename for the figure.
    `caption`: Caption for the figure.

    Parameters for df.plot():
    `x`: Column name for x-axis (index by default).
    `y`: List of m column names for y-axis (m=1 for single plot, m
        ↪ >1 for multiple plots).
    `kind`: only bar is allowed.
    `logx` / `logy` (bool): log scale for x/y axis.

    `y_ci`: Confidence intervals for errorbars.
        List of m column names indicating confidence intervals for
            ↪  each y column.
        Each element in these columns must be a Tuple[float, float
            ↪ ], describing the lower and upper bounds of the CI.

     `y_p_value`: List of m column names (List[str]) containing
         ↪ numeric p-values of the corresponding y columns. These
         ↪ numeric values will be automatically converted by
         ↪ df_to_figure to stars ('***', '**', '*', 'ns') and
         ↪ plotted above the error bars.

    If provided, the length of `y_ci`, and `y_p_value` should be
        ↪ the same as of `y`.

    Example:
    Suppose, we have:

    df_lin_reg_longevity = pd.DataFrame({
```

```
    'adjusted_coef': [0.4, ...], 'adjusted_coef_ci': [(0.35,
        ↪ 0.47), ...], 'adjusted_coef_pval': [0.012, ...],
    'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
        ↪ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
        ↪ [0.0001, ...],
}, index=['var1', ...])

then:
df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
    ↪ caption='Coefficients of ...', kind='bar',
    y=['adjusted_coef', 'unadjusted_coef'],
    y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
    y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval'])
"""
```

C.3 Code Description

The analysis undertakes several steps to evaluate the relationships between message attributes in the dataset from the File: `data_datamix_tier_0_train_interleaved_n100.csv` The objective is to explore syntactic complexity's impact on message length, utilizing descriptive statistics, correlation measures, and regression analysis. Below is a high-level explanation of each step:

C.4 Descriptive Statistics

The first analytical phase involves computing descriptive statistics to characterize the dataset in terms of message length and syntactic complexity. For this purpose, the code extracts "user" and "assistant" messages from a parsed JSON structure within the data column. It then computes the length of these messages and assesses syntactic complexity by identifying arithmetic operations within the user message content. The average (mean), standard deviation (std), and count of the message lengths and complexity are recorded, highlighting the central tendency and variability in the dataset.

C.5 Correlation Analysis

The correlation analysis focuses on assessing the linear relationship between syntactic complexity and the length of the assistant's response. This step employs both Pearson and Spearman's rank correlation coefficients, providing insights into linear and monotonic relationships between these variables, respectively. The results, including correlation coefficients and associated p-values, ascertain if increased complexity corresponds with longer assistant responses.

Created by data-to-paper (AI)

Created by data-to-paper (AI)

C.6 Regression Analysis

The regression analysis section models the relationship between syntactic complexity, user message length, and the resulting assistant response length, using multiple linear regression. The regression formula incorporates an interaction term to explore how complexity and user message length collectively influence response length. The results from the Ordinary Least Squares (OLS) model include coefficient estimates, standard errors, and confidence intervals, providing a detailed understanding of each term's effect on the response variable. P-values are also reported to determine the statistical significance of these effects.

C.7 Visualization and Additional Results

Subsequent visualization of the regression results in a bar plot graphically presents the estimated effects of syntactic complexity on response length with confidence intervals. Additional outputs, such as the total number of observations and correlation coefficients, are saved for reference in an external file, enabling further review and synthesis of findings.

This structured approach blends descriptive and inferential statistics to thoroughly examine and visualize the relationships of interest within the dataset.

C.8 Code Output

df_complexity_effect.pkl

```
                        Estimate  Std Err t value      P>|t| CI
                            Lower CI Upper
Term
Intercept                  157.3    3.345   47.02          0
    150.8    163.9
complexity                -7.183   0.4875  -14.73   3.24e-48
    -8.139   -6.227
user_length               -2.012   0.0723  -27.83  1.63e-159
    -2.154   -1.871
complexity:user_length   0.1101 0.002269   48.52          0
    0.1057   0.1146
```

df_correlation.pkl

```
Pearson Correlation
    Pearson p-value
    Spearman Correlation
    Spearman p-value
```

Created by data-to-paper (AI)

```
Syntactic Complexity vs Response Length              -0.546
                  0              -0.6206                 0
```

df_desc_stat.pkl

```
                   mean   std count
user_length       93.42 32.66  5454
assistant_length   12.7 24.02  5454
complexity        8.186 5.023  5454
```

df_regression.pkl

```
                       Estimate  Std Err CI Lower CI Upper    p-
                          value
Intercept                 157.3    3.345    150.8    163.9
            0
complexity               -7.183   0.4875   -8.139   -6.227   3.24e
    -48
user_length              -2.012   0.0723   -2.154   -1.871  1.63e
    -159
complexity:user_length   0.1101 0.002269   0.1057   0.1146
            0
```

additional_results.pkl

```
{
    'Total number of observations': 5454,
    'Pearson correlation coefficient': -0.546,
    'Spearman correlation coefficient': -0.6206,
}
```

D LaTeX Table Design

D.1 Code

The LaTeX Table Design was carried out using the following custom code:

```
# IMPORT
import pandas as pd
from my_utils import df_to_latex, df_to_figure, is_str_in_df,
    ↪ split_mapping, AbbrToNameDef

# PREPARATION FOR ALL TABLES AND FIGURES
all_mapping: AbbrToNameDef = {
    'Estimate': ('Estimate', 'The estimated coefficient value from
        ↪  the regression model.'),
    'Standard Error': ('Std Err', 'The standard error of the
        ↪ coefficient estimate.'),
```

Created by data-to-paper (AI)

```
    'CI Lower': ('CI Lower', 'The lower bound of the 95%
        ↪ confidence interval for the estimate.'),
    'CI Upper': ('CI Upper', 'The upper bound of the 95%
        ↪ confidence interval for the estimate.'),
    'P>|t|': ('P-value', 'The p-value for testing the null
        ↪ hypothesis that the coefficient is zero.'),
    'complexity': ('Complexity', 'Number of arithmetic operations
        ↪ in the user input.'),
    'user_length': ('User Length', 'Length of the user message in
        ↪ characters.'),
    'assistant_length': ('Assistant Length', 'Length of the
        ↪ assistant message in characters.'),
    'Pearson Correlation': ('Pearson r', 'Pearson correlation
        ↪ coefficient.'),
    'Pearson p-value': ('Pearson p', 'P-value for the Pearson
        ↪ correlation.'),
    'Spearman Correlation': ('Spearman rho', 'Spearman correlation
        ↪  coefficient.'),
    'Spearman p-value': ('Spearman p', 'P-value for the Spearman
        ↪ correlation.'),
    'Syntactic Complexity vs Response Length': ('Complexity vs
        ↪ Response', None),
    'complexity:user_length': ('Complexity * User Length', '
        ↪ Interaction term between Complexity and User Length'),
}

# Process df_complexity_effect
df_complexity_effect = pd.read_pickle('df_complexity_effect.pkl')

# Format values:
# Not Applicable

# Remove the Intercept row
df_complexity_effect = df_complexity_effect.drop(index='Intercept'
    ↪ )

# Rename rows and columns:
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_complexity_effect, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_complexity_effect.rename(columns=abbrs_to_names, index=
    ↪ abbrs_to_names, inplace=True)

df_to_figure(
    df_complexity_effect, 'df_complexity_effect_formatted',
    caption="Effect of Syntactic Complexity on Response Length",
    note="Plot shows the estimated coefficients with interaction
        ↪ term on assistant message length.",
    glossary=glossary,
```

Created by data-to-paper (AI)

```
    kind='bar',
    y=['Estimate'],
    y_ci=[('CI Lower', 'CI Upper')],
    y_p_value=['P-value']
)


# Process df_correlation
df_correlation = pd.read_pickle('df_correlation.pkl')

# Format values:
# Not Applicable

# Rename rows and columns and transpose to make the table narrower
    ↪ :
df_correlation = df_correlation.T
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_correlation, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_correlation.rename(columns=abbrs_to_names, index=abbrs_to_names
    ↪ , inplace=True)

df_to_latex(
    df_correlation, 'df_correlation_formatted',
    caption="Correlation between Syntactic Complexity and Response
        ↪  Length",
    glossary=glossary
)


# Process df_desc_stat
df_desc_stat = pd.read_pickle('df_desc_stat.pkl')

# Format values:
# Not Applicable

# Rename rows and columns:
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_desc_stat, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_desc_stat.rename(columns=abbrs_to_names, index=abbrs_to_names,
    ↪ inplace=True)

df_to_latex(
    df_desc_stat, 'df_desc_stat_formatted',
    caption="Descriptive Statistics of Message Length and
        ↪ Complexity",
    glossary=glossary
)
```

Created by data-to-paper (AI)

```python
# Process df_regression
df_regression = pd.read_pickle('df_regression.pkl')

# Format values:
# Not Applicable

# Drop the Intercept row to avoid including it in the scientific
    ↪ table
df_regression = df_regression.drop(index='Intercept')

# Rename rows and columns:
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_regression, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_regression.rename(columns=abbrs_to_names, index=abbrs_to_names,
    ↪  inplace=True)

# Transpose the DataFrame to make it narrower
df_regression = df_regression.T

df_to_latex(
    df_regression, 'df_regression_formatted',
    caption="Regression Analysis with Interaction Term for
        ↪ Complexity and User Message Length",
    glossary=glossary
)
```

D.2 Provided Code

The code above is using the following provided functions:

```python
def df_to_latex(df,
        filename: str, caption: str,
        note: str = None,
        glossary: Dict[Any, str] = None,
    ):
    """
    Saves a DataFrame `df` and creates a LaTeX table.
    `filename`, `caption`: as in `df.to_latex`.
    `note`: Note to be added below the table caption.
    `glossary`: Glossary for the table.
    """


def df_to_figure(
        df, filename: str, caption: str,
```

Created by data-to-paper (AI)

```
    note: str = None, glossary: Dict[Any, str] = None,
    x: Optional[str] = None, y: List[str] = None,
    kind: str = 'bar',
    logx: bool = False, logy: bool = False,
    y_ci: Optional[List[str]] = None,
    y_p_value: Optional[List[str]] = None,
    xlabel: str = None, ylabel: str = None,
):
"""
Save a DataFrame `df` and create a LaTeX figure.
Parameters, for LaTex embedding of the figure:
`filename`: Filename for the figure.
`caption`: Caption for the figure.
`note`: Note to be added below the figure caption.
`glossary`: Glossary for the figure.

Parameters for df.plot():
`x`: Column name for x-axis (index by default).
`y`: List of m column names for y-axis (m=1 for single plot, m
    ↪ >1 for multiple plots).
`kind`: only bar is allowed.
`logx` / `logy` (bool): log scale for x/y axis.
`xlabel`: Label for the x-axis.
`ylabel`: Label for the y-axis.

`y_ci`: Confidence intervals for errorbars.
    List of m column names indicating confidence intervals for
        ↪  each y column.
    Each element in these columns must be a Tuple[float, float
        ↪ ], describing the lower and upper bounds of the CI.

 `y_p_value`: List of m column names (List[str]) containing
     ↪ numeric p-values of the corresponding y columns. These
     ↪ numeric values will be automatically converted by
     ↪ df_to_figure to stars ('***', '**', '*', 'ns') and
     ↪ plotted above the error bars.

If provided, the length of `y_ci`, and `y_p_value` should be
    ↪ the same as of `y`.

Example:
Suppose, we have:

df_lin_reg_longevity = pd.DataFrame({
    'adjusted_coef': [0.4, ...], 'adjusted_coef_ci': [(0.35,
        ↪ 0.47), ...], 'adjusted_coef_pval': [0.012, ...],
    'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
        ↪ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
        ↪ [0.0001, ...],
```

Created by data-to-paper (AI)

```
    }, index=['var1', ...])

    then:
    df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
        ↪ caption='Coefficients of ...', kind='bar',
        y=['adjusted_coef', 'unadjusted_coef'],
        y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
        y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval'])
    """


def is_str_in_df(df: pd.DataFrame, s: str):
    return any(s in level for level in getattr(df.index, 'levels',
        ↪  [df.index]) + getattr(df.columns, 'levels', [df.columns
        ↪ ]))

AbbrToNameDef = Dict[Any, Tuple[Optional[str], Optional[str]]]

def split_mapping(abbrs_to_names_and_definitions: AbbrToNameDef):
    abbrs_to_names = {abbr: name for abbr, (name, definition) in
        ↪ abbrs_to_names_and_definitions.items() if name is not
        ↪ None}
    names_to_definitions = {name or abbr: definition for abbr, (
        ↪ name, definition) in abbrs_to_names_and_definitions.
        ↪ items() if definition is not None}
    return abbrs_to_names, names_to_definitions
```

D.3 Code Output

df_complexity_effect_formatted.pkl

```
\begin{figure}[htbp]
\centering
\includegraphics{df_complexity_effect_formatted.png}
\caption{Effect of Syntactic Complexity on Response Length
Plot shows the estimated coefficients with interaction term on
    assistant message length.
Estimate: The estimated coefficient value from the regression
    model.
CI Lower: The lower bound of the 95\% confidence interval for the
    estimate.
CI Upper: The upper bound of the 95\% confidence interval for the
    estimate.
P-value: The p-value for testing the null hypothesis that the
    coefficient is zero.
Complexity: Number of arithmetic operations in the user input.
User Length: Length of the user message in characters.
```

Created by data-to-paper (AI)

```
Complexity * User Length: Interaction term between Complexity and
    User Length.
Significance: ns p $>$= 0.01, * p $<$ 0.01, ** p $<$ 0.001, *** p
    $<$ 0.0001.}
\label{figure:df-complexity-effect-formatted}
\end{figure}
% This latex figure presents "df_complexity_effect_formatted.png",
% which was created from the df:
%
% index,"Estimate","Std Err","t value","P-value","CI Lower","CI
    Upper"
% "Complexity",-7.183,0.4875,-14.73,<1e-06,-8.139,-6.227
% "User Length",-2.012,0.0723,-27.83,<1e-06,-2.154,-1.871
% "Complexity * User Length",0.1101,0.002269,48.52,<1e-06,0.1057,
    0.1146
%
% To create the figure, this df was plotted with the command:
%
% df.plot(kind='bar', y=['Estimate'])
%
% Confidence intervals for y-values were then plotted based on
    column: [('CI Lower', 'CI Upper')].
%
% P-values for y-values were taken from column: ['P-value'].
%
% These p-values were presented above the data points as stars (
    with significance threshold values indicated in the figure
    caption).
```

df_correlation_formatted.pkl

```
\begin{table}[h]
\caption{Correlation between Syntactic Complexity and Response
    Length}
\label{table:df-correlation-formatted}
\begin{threeparttable}
\renewcommand{\TPTminimum}{\linewidth}
\makebox[\linewidth]{%
\begin{tabular}{lr}
\toprule
 & Complexity vs Response \\
\midrule
\textbf{Pearson r} & -0.546 \\
\textbf{Pearson p} & 0 \\
\textbf{Spearman rho} & -0.6206 \\
\textbf{Spearman p} & 0 \\
\bottomrule
\end{tabular}}
\begin{tablenotes}
```

Created by data-to-paper (AI)

```
\footnotesize
\item \textbf{Pearson r}: Pearson correlation coefficient.
\item \textbf{Pearson p}: P-value for the Pearson correlation.
\item \textbf{Spearman rho}: Spearman correlation coefficient.
\item \textbf{Spearman p}: P-value for the Spearman correlation.
\end{tablenotes}
\end{threeparttable}
\end{table}
```

df_desc_stat_formatted.pkl

```
\begin{table}[h]
\caption{Descriptive Statistics of Message Length and Complexity}
\label{table:df-desc-stat-formatted}
\begin{threeparttable}
\renewcommand{\TPTminimum}{\linewidth}
\makebox[\linewidth]{%
\begin{tabular}{lrrr}
\toprule
 & mean & std & count \\
\midrule
\textbf{User Length} & 93.42 & 32.66 & 5454 \\
\textbf{Assistant Length} & 12.7 & 24.02 & 5454 \\
\textbf{Complexity} & 8.186 & 5.023 & 5454 \\
\bottomrule
\end{tabular}}
\begin{tablenotes}
\footnotesize
\item \textbf{Complexity}: Number of arithmetic operations in the
    user input.
\item \textbf{User Length}: Length of the user message in
    characters.
\item \textbf{Assistant Length}: Length of the assistant message
    in characters.
\end{tablenotes}
\end{threeparttable}
\end{table}
```

df_regression_formatted.pkl

```
\begin{table}[h]
\caption{Regression Analysis with Interaction Term for Complexity
    and User Message Length}
\label{table:df-regression-formatted}
\begin{threeparttable}
\renewcommand{\TPTminimum}{\linewidth}
\makebox[\linewidth]{%
\begin{tabular}{llll}
\toprule
```

Created by data-to-paper (AI)

```
 & Complexity & User Length & Complexity * User Length \\
\midrule
\textbf{Estimate} & -7.183 & -2.012 & 0.1101 \\
\textbf{Std Err} & 0.4875 & 0.0723 & 0.002269 \\
\textbf{CI Lower} & -8.139 & -2.154 & 0.1057 \\
\textbf{CI Upper} & -6.227 & -1.871 & 0.1146 \\
\textbf{p-value} & $<$1e-06 & $<$1e-06 & $<$1e-06 \\
\bottomrule
\end{tabular}}
\begin{tablenotes}
\footnotesize
\item \textbf{Estimate}: The estimated coefficient value from the
    regression model.
\item \textbf{CI Lower}: The lower bound of the 95\% confidence
    interval for the estimate.
\item \textbf{CI Upper}: The upper bound of the 95\% confidence
    interval for the estimate.
\item \textbf{Complexity}: Number of arithmetic operations in the
    user input.
\item \textbf{User Length}: Length of the user message in
    characters.
\item \textbf{Complexity * User Length}: Interaction term between
    Complexity and User Length
\end{tablenotes}
\end{threeparttable}
\end{table}
```